

LAPORAN PRATIKUM PEKAN 6

STRUKTUR DATA

Double Linked List



Disusun oleh:

Jose Moniz de Araujo 2511538002

Dosen pengampu:

Wahyudi Dr. S.T.M.T

Asisten Pratikum:

Sherly Sukmadira

**DEPARTAMENT INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

TAHUN

2026

KATA PENGATAR

Saya ingin menyampaikan rasa syukur kepada Tuhan Yang Maha Esa, karena berkat -Nya saya dapat menyelesaikan laporan praktik ini dengan sukses dan tepat waktu. Laporan ini disusun sebagai bagian dari kegiatan praktikum mata kuliah Struktur Data, dengan fokus pada topik **Double Linked List**.

Melalui praktikum ini, saya mempelajari konsep dasar Double Linked List, cara kerja node dengan dua pointer, dan implementasi operasi seperti penambahan, penghapusan dan pencarian data. Praktikum ini juga membantu saya memahami bagaimana struktur data digunakan dalam pemrograman untuk mengelola data secara lebih efisien.

Saya menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, saya sangat mengharagai kritik dan saran untuk perbaikan di masa mendatang.

Saya ingin mengucapkan terima kasih kepada dosen dan semua pihak yang membantu dalam pelaksanaan praktikum ini. Saya berharap laporan ini bermanfaat bagi pembaca dan meningkatkan pemahaman mereka tentang topik Double Linked List.

Padang, 13 Mei 2026

Jose Moniz de Araujo

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan terkini dalam teknologi informasi dan pemrograman menuntun manajemen data yang cepat dan efisien. Dalam dunia pemrograman, struktur data merupakan komponen penting yang digunakan untuk mengatur dan menyimpan data agar dapat diproses secara efisien. Salah satu struktur data yang sering digunakan adalah Linked List.

Double Linked List merupakan perluasan dari Single Linked List, dengan dua pointer pada setiap node: satu menunjuk ke node berikutnya dan satu menunjuk ke node sebelumnya. Koneksi dua arah ini memungkinkan pengambilan dan manipulasi data yang lebih fleksibel dan efisien daripada Single Linked List.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar Double Linked List, cara kerja setiap node, dan implementasi operasi dasar seperti penambahan, penghapusan, dan pencarian data. Selain itu, praktikum ini juga bertujuan untuk mengasah keterampilan logika pemrograman mahasiswa dan pemahaman tentang implementasi struktur data dalam bahasa pemrograman. Dengan mempelajari Double Linked List, mahasiswa akan memahami pentingnya penggunaan struktur data dalam menyelesaikan berbagai masalah pemrograman secara efektif dan terstruktur.

1.2 Tujuan Praktikum

1. Memahami konsep dasar struktur data Double Linked List.
2. Memahami cara kerja node yang memiliki pointer ke node berikutnya dan sebelumnya.
3. Mengimplementasikan operasi dasar pada Double Linked List, seperti penambahan, penghapusan, dan pencarian

4. Melatih keterampilan logika pemrograman dalam menerapkan struktur data.

1.3 Manfaat Pratikum

1. Meningkatkan pemahaman tentang struktur data Double Linked List.
2. Meningkatkan kemampuan untuk membuat program menggunakan struktur data.
3. Membantu memahami proses manajemen data yang lebih fleksibel dan efisien.
4. Menjadi dasar untuk mempelajari struktur data dan algoritma yang lebih canggih.

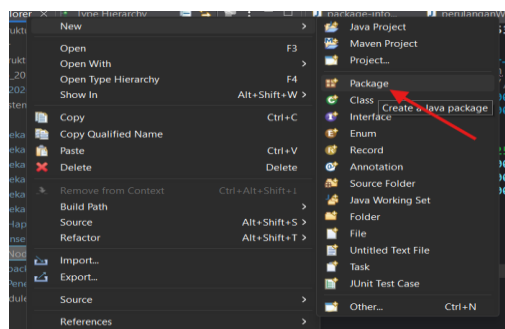
BAB II

PEMBAHASAN

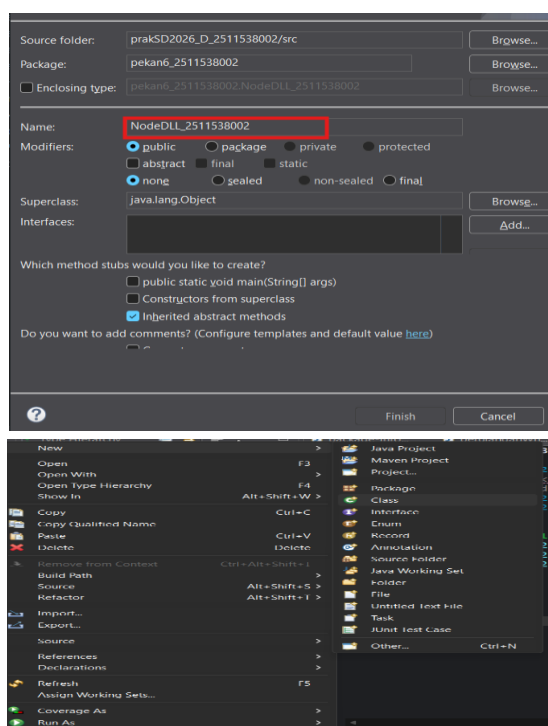
2.1 Kode Program Pratikum pekan 6

2.1.1 NodeDLL_2511538002

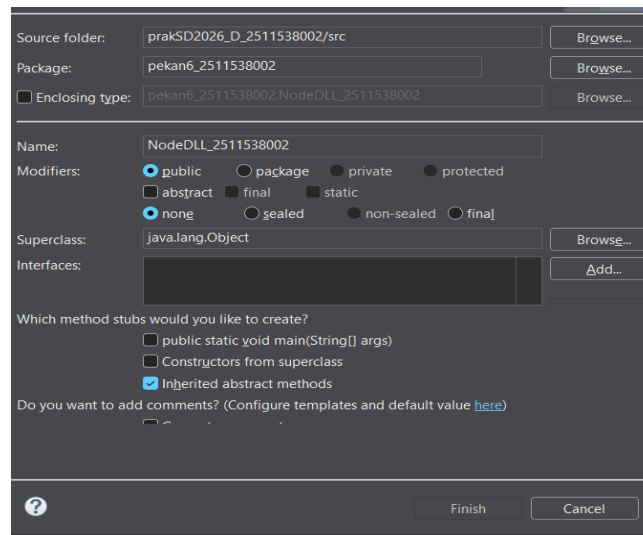
1. Buka aplikasi, lalu buat package baru pada src klik kanan, lalu pilih “New” dan klik Tulis package.



2. Kemudian buat nama packagenya tanpa spasi, huruf kapital, karakter khusus. Beri nama “pekan6_2511538002”. Lalu klik finish.
3. Ketika package sudah jadi. Klik pada bagian Pekan6_251153800”. Lalu klik “New” dan pilih bagian “Class” untuk memulai membuat program.



4. buat nama program yang akan dibuat pada bagian nama tanpa spasi dan menggunakan huruf kapital pada awal kata, lalu klik bagian “public static void main(String[]args)”. Kemudian klik “finish”.



5. Masukkan Syntax berikut

```
1 package pekan6_2511538002;
2
3 public class NodeDLL_2511538002 {
4     //mendefinisikan kelas Node
5     int data_8002; //data
6     NodeDLL_2511538002 next_8002; // pointer ke next node
7     NodeDLL_2511538002 prev_8002; // pointer ke previous node
8
9     //konstruktor
10    public NodeDLL_2511538002 (int data_8002) {
11        this.data_8002= data_8002;
12        this.next_8002 = null;
13        this.prev_8002 = null;
14    }
15 }
16
17
18 }
19
```

2.1.2 InsertDLL_25115380

1. Ulangi pembuatan class seperti program sebelumnya dan beri nama Class tersebut “InsertDLL_2511538002”. Lalu masukkan Syntax seperti berikut:

```

1 package pekan6_2511538002;
2
3 public class InsertDLL_2511538002 {
4     //memasukkan node di awal DLL
5
6     static NodeDLL_2511538002 insertBegin_8002(NodeDLL_2511538002 head, int data_8002) {
7         // buat node baru
8         NodeDLL_2511538002 new_node = new NodeDLL_2511538002 (data_8002);
9         // jadikan pointer nextnya head
10
11         new_node.next_8002 = head;
12         // jadikan pointer prev head di new_node
13         if (head != null) {
14             head.prev_8002 = new_node;
15         }
16         return new_node;
17     }
18     // fungsi memasukkan node di akhir
19
20     public static NodeDLL_2511538002 insertEnd_8002(NodeDLL_2511538002 head, int new_data) {
21         // buat node baru
22         NodeDLL_2511538002 new_node = new NodeDLL_2511538002 (new_data);
23         // jika DLL null jadikan head
24         if (head == null) {
25             head = new_node;
26         }
27         else {
28             NodeDLL_2511538002 curr = head;
29             while (curr.next_8002 != null) {
30                 curr = curr.next_8002;
31             }
32             curr.next_8002 = new_node;
33             new_node.prev_8002 = curr;
34         }
35         return head;
36     }
37     // fungsi memasukkan node di posisi tertentu
38     public static NodeDLL_2511538002 insertAtPosition_8002(NodeDLL_2511538002 head, int pos, int new_data) {
39         // buat node baru
40         NodeDLL_2511538002 new_node = new NodeDLL_2511538002 ( new_data);
41         if (pos == 1) {
42             new_node.next_8002 = head;
43             if (head != null) {
44                 head.prev_8002 = new_node;
45             }
46             head = new_node;
47             return head;
48         }
49         NodeDLL_2511538002 curr = head;
50         for (int i = 1; i < pos - 1; i++) {
51             curr = curr.next_8002;
52         }
53         if (curr == null) {
54             System.out.println("posisi tidak ada");
55             return head;
56         }
57
58         new_node.prev_8002 = curr;
59         new_node.next_8002 = curr.next_8002;
60         curr.next_8002 = new_node;
61         if (new_node.next_8002 != null) {
62             new_node.next_8002.prev_8002 = new_node;
63         }
64         return head;
65     }
66     public static void printList_8002(NodeDLL_2511538002 head) {
67         NodeDLL_2511538002 curr = head;
68         while (curr != null) {
69             System.out.print(curr.data_8002 + " <-> ");
70             curr = curr.next_8002;
71         }
72         System.out.println();
73     }
74
75     public static void main(String[] args) {
76         // membuat DLL 2 <-> 3 <-> 5
77         NodeDLL_2511538002 head = new NodeDLL_2511538002(2);
78         head.next_8002 = new NodeDLL_2511538002(3);
79         head.next_8002.prev_8002 = head;
80         head.next_8002.next_8002 = new NodeDLL_2511538002(5);
81         head.next_8002.next_8002.prev_8002 = head.next_8002;
82         // cetak dll awal
83         System.out.print("DLL awal: ");
84         printList_8002(head);
85         // tambah 1 di awal
86         head = insertBegin_8002(head, 1);
87         System.out.print("simpul 1 ditambah di awal: ");
88         printList_8002(head);
89         //tambah 6 di akhir
90         System.out.print("simpul 6 ditambah di akhir: ");
91         int data_8002 = 6;
92         head = insertEnd_8002(head, data_8002);
93         printList_8002(head);
94         // masukkan node 4 di posisi 4
95         System.out.print("tambah node 4 di posisi 4: ");
96         int data2 = 4;
97         int pos = 4;
98         head = insertAtPosition_8002(head, pos, data2);
99         printList_8002(head);
100
101
102     }
103
104 }
105
106

```

1. Kemudian klik Run untuk melihat hasil dari syntax yang sudah dibuat :

```
DLL awal: 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

2.1.3 PenelusuranDLL_2511538002

1. Ulangi pembuatan Class program sebelumnya dan beri nama Class tersebut “PenelusuranDLL_2511538002”. Lalu masukkan syntax seperti berikut.

```
1 package pekan6_2511538002;
2
3 public class PenelusuranDLL_2511538002 {
4     // fungsi penelusuran maju
5     static void forwardTraversal_8002(NodeDLL_2511538002 head) {
6         //mulai penelusuran dari head
7
8         //mulai penelusuran dari head
9         NodeDLL_2511538002 curr = head;
10        //lanjutkan sampai akhir
11        while (curr != null) {
12            // print data
13            System.out.print(curr.data_8002 + " <-> ");
14            // pindah ke node berikutnya
15            curr = curr.next_8002;
16        }
17        // print spasi
18        System.out.println();
19    }
20
21    // fungsi penelusuran mundur
22    static void backwardTraversal_8002(NodeDLL_2511538002 tail) {
23        // mulai dari akhir
24        NodeDLL_2511538002 curr = tail;
25        // lanjut sampai head
26        while (curr != null) {
27            // cetak data
28            System.out.print(curr.data_8002 + " <-> ");
29            // pindah ke node sebelumnya
30            curr = curr.prev_8002;
31        }
32        //cetak spasi
33        System.out.println();
34    }
35
36
37    public static void main(String[] args) {
38        // cetak DLL
39        NodeDLL_2511538002 head = new NodeDLL_2511538002(1);
40        NodeDLL_2511538002 second = new NodeDLL_2511538002(2);
41        NodeDLL_2511538002 third = new NodeDLL_2511538002(3);
42        head.next_8002 = second;
43        second.prev_8002 = head;
44        second.next_8002 = third;
45        third.prev_8002 = second;
46
47        System.out.println("penelusuran maju:");
48        forwardTraversal_8002(head);
49
50        System.out.println("penelusuran mundur:");
51        backwardTraversal_8002(third);
52
53    }
54
55 }
56
```

2. Klik “Run” untuk melihat hasil syntax yang sudah dibuat:

```
<terminated> PenelusuranDLL_251
penelusuran maju:
1 <-> 2 <-> 3 <->
penelusuran mundur:
3 <-> 2 <-> 1 <->
```


2.1.4 HapusDLL_2511538002

1. Ulangi pembuatan Class seperti program sebelumnya dan beri nama Class tersebut “HapusDLL_2511538002”. Lalu masukkan syntax seperti berikut:

```
1 //hapus DLL_2511538002;
2 public class HapusDLL_2511538002 {
3
4     // fungsi menghapus node awal
5     public static NodeDLL_2511538002 delHead(NodeDLL_2511538002 head) {
6         if (head == null) {
7             return null;
8         }
9         NodeDLL_2511538002 temp = head;
10        head = head.next_8002;
11        if (head != null) {
12            head.prev_8002 = null;
13        }
14        return head;
15    }
16
17    // fungsi menghapus di akhir
18    public static NodeDLL_2511538002 delLast(NodeDLL_2511538002 head, int pos) {
19        if (head == null) {
20            return null;
21        }
22        NodeDLL_2511538002 curr = head;
23        // traverse sampai ke node
24        for (int i = 1; curr != null && i < pos; ++i) {
25            curr = curr.next_8002;
26        }
27        // jika sudah sampai
28        if (curr == null) {
29            return head;
30        }
31        // update pointer
32        if (curr.prev_8002 != null) {
33            curr.prev_8002.next_8002 = curr.next_8002;
34        }
35        if (curr.next_8002 != null) {
36            curr.next_8002.prev_8002 = curr.prev_8002;
37        }
38        // jika yang
39        if (head == curr) {
40            head = curr.next_8002;
41        }
42        return head;
43    }
44
45    // fungsi mencetak
46    public static void printList(NodeDLL_2511538002 head) {
47        NodeDLL_2511538002 curr = head;
48        while (curr != null) {
49            System.out.print(curr.data_8002 + " ");
50            curr = curr.next_8002;
51        }
52        System.out.println();
53    }
54
55    // fungsi menghapus di akhir
56    public static NodeDLL_2511538002 delLast(NodeDLL_2511538002 head) {
57        if (head == null) {
58            return null;
59        }
60        if (head.next_8002 == null) {
61            return null;
62        }
63        NodeDLL_2511538002 curr = head;
64        while (curr.next_8002 != null) {
65            curr = curr.next_8002;
66        }
67        // update [pointer previous node
68        if (curr.prev_8002 != null) {
69            curr.prev_8002.next_8002 = null;
70        }
71        return head;
72    }
73
74    // fungsi menghapus node posisi tertentu
75    public static NodeDLL_2511538002 delPos(NodeDLL_2511538002 head, int pos) {
76        if (head == null) {
77            return null;
78        }
79        NodeDLL_2511538002 curr = head;
80        // traverse
81        for (int i = 1; curr != null && i < pos; ++i) {
82            curr = curr.next_8002;
83        }
84        // jika pos
85        if (curr == null) {
86            return head;
87        }
88        // update
89        if (curr.prev_8002 != null) {
90            curr.prev_8002.next_8002 = curr.next_8002;
91        }
92        // jika yang
93        if (head == curr) {
94            head = curr.next_8002;
95        }
96        return head;
97    }
98
99    // fungsi
100    public static void printList(NodeDLL_2511538002 head) {
101        NodeDLL_2511538002 curr = head;
102        while (curr != null) {
103            System.out.print(curr.data_8002 + " <-> ");
104            curr = curr.next_8002;
105        }
106        System.out.println();
107    }
108
109    public static void main(String[] args) {
110        // buat sebuah dll
111        NodeDLL_2511538002 head = new NodeDLL_2511538002(1);
112        head.next_8002 = new NodeDLL_2511538002(2);
113        head.next_8002.prev_8002 = head;
114        head.next_8002.next_8002 = new NodeDLL_2511538002(3);
115        head.next_8002.next_8002.prev_8002 = head.next_8002;
116        head.next_8002.next_8002.next_8002 = new NodeDLL_2511538002(4);
117        head.next_8002.next_8002.next_8002.prev_8002 = head.next_8002.next_8002;
118        head.next_8002.next_8002.next_8002.next_8002 = new NodeDLL_2511538002(5);
119        head.next_8002.next_8002.next_8002.next_8002.prev_8002 = head.next_8002.next_8002.next_8002;
120        System.out.print("DLL Awal: ");
121        printList(head);
122        System.out.print("setelah head dihapus: ");
123        head = delHead(head);
124        printList(head);
125        System.out.print("setelah node terakhir di hapus: ");
126        head = delLast(head);
127        printList(head);
128        System.out.print("menghapus node ke 2: ");
129        head = delPos(head, 2);
130        printList(head);
131    }
132 }
```

2. Klik “Run” untuk melihat hasil syntax yang sudah dibuat:

```
DLL Awal: 1 <-> 2 <-> 3 <-> 4 <-> 5 <->
setelah head dihapus: 2 <-> 3 <-> 4 <-> 5 <->
setelah node terakhir di hapus: 2 <-> 3 <-> 4 <->
menghapus node ke 2: 2 <-> 4 <->
```

2.2 Tugas Pratikum pekan6_2511538002

2.2.1 Lagu_2511538002

1. Ulangi pembuatan Class seperti program sebelumnya dan beri nama Class tersebut “Lagu_2511538002”. Lalu masukkan syntax seperti berikut:

```
1 | prakSD2026_D_2511538002/src/pekan6_2511538002/Lagu_2511538002.java
2 |
3 | public class Lagu_2511538002 {
4 |     //atribut lagu
5 |     private String judul_8002;
6 |     private String penyanyi_8002;
7 |
8 |     //pointer ke node berikutnya dan sebelum
9 |     Lagu_2511538002 next_8002;
10 |    Lagu_2511538002 prev_8002;
11 |
12 |    //konstruktor untuk mengisi data lagu
13 |    public Lagu_2511538002(String judul_8002,String penyanyi_8002) {
14 |        this.judul_8002 = judul_8002;
15 |        this.penyanyi_8002 = penyanyi_8002;
16 |    }
17 |    //pointer awal bernilai null
18 |    this.next_8002 = null;
19 |    this.prev_8002 = null;
20 | }
21 | //getter judul dan penyanyi
22 | public String getJudul_8002() {
23 |     return judul_8002;
24 | }
25 | public String getPenyanyi_8002() {
26 |     return penyanyi_8002;
27 | }
28 | //sette judul dan penyanyi
29 | public void setJudul(String judul_8002) {
30 |     this.judul_8002 =judul_8002;
31 | }
32 | public void setPenyanyi(String penyanyi_8002) {
33 |     this.penyanyi_8002 = penyanyi_8002;
34 | }
35 | }
36 |
```

2.2.2 Penjelasan kode program Lagu_2511538002

1. Atribut

```
private String judul_8002;
private String
penyanyi_8002;
```

Atribut ini digunakan untuk menyimpan informasi lagu, yaitu: menyimpan judul lagu dan menyimpan nama penyanyi.

2. Pointer Double Linked List

```
Lagu_2511538002 next_8002;  
Lagu_2511538002 prev_8002;
```

Pointer digunakan untuk menghubungkan node satu dengan node lainnya pada double linked list

3. Constructor

```
Lagu_2511538002(String judul_8002,String  
penyanyi_8002) {
```

Constructor digunakan untuk menginisialisasi objek saat lagu dibuat. Konstruktor akan mengisi nilai judul dan penyanyi sesuai input pengguna.

4. Inisialisasi data pada Constructor

```
this.judul_8002 = judul_8002;  
this.penyanyi_8002 = penyanyi_8002;
```

Syntax ini digunakan untuk bisa memasukkan nilai parameter ke dalam atribut objek menggunakan keyword this.

5. Inisialisasi Pointer

```
this.next_8002 = null;  
this.prev_8002 = null;
```

Pointer ini diatur bernilai null karena pada saat node pertama dibuat, node tersebut belum terhubung dengan node lain.

6. Getter Method

```
public String getJudul_8002()  
public String getPenyanyi_8002()
```

Getter digunakan untuk bisa mengambil data judul dan data penyanyi atau bisa menampilkan nilai atribut dari class.

7. Setter Method

```
public void setJudul(String judul_8002)  
public void setPenyanyi(String penyanyi_8002)
```

Setter digunakan untuk mengubah nilai atribut pada objek judul lagu dan nama Penyanyi.

2.2.3 Kode program Musik_2511538002

- 1 Ulangi pembuatan Class seperti program sebelumnya dan beri nama Class tersebut “Musik_2511538002”. Lalu masukkan syntax seperti berikut:

```

1 package kpkane_2511538002;
2 import java.util.Scanner;
3
4 public class Musik_2511538002 {
5
6     //pointer awal dan akhir double Linked List
7     Lagu_2511538002 head_8002;
8     Lagu_2511538002 tail_8002;
9
10    //konstruktor
11    public Musik_2511538002() {
12        head_8002 = null;
13        tail_8002 = null;
14    }
15
16    //method menambah lagu di akhir playlist
17    public void tambahLagu_8002(String judul_8002, String penyanyi_8002) {
18
19        //membuat node lagu baru
20        Lagu_2511538002 laguBaru_8002 = new Lagu_2511538002(judul_8002, penyanyi_8002);
21
22        // jika playlist masih kosong
23        if (head_8002 == null) {
24            head_8002 = laguBaru_8002;
25            tail_8002 = laguBaru_8002;
26        } else {
27
28            //menyambungkan node baru ke akhir list
29            tail_8002.next_8002 = laguBaru_8002;
30            laguBaru_8002.prev_8002 = tail_8002;
31
32            //memindahkan tail ke node baru
33            tail_8002 = laguBaru_8002;
34        }
35        System.out.println("Lagu berhasil ditambahkan!");
36    }
37
38    //method menghapus lagu pertama
39    public void hapusLaguAwal_8002() {
40
41        //cek jika playlist kosong
42        if (head_8002 == null) {
43            System.out.println("Playlist kosong!");
44            return;
45        }
46
47        // jika hanya ada satu lagu
48        if (head_8002 == tail_8002) {
49            head_8002 = null;
50            tail_8002 = null;
51        } else {
52
53            //pindahkan head ke node berikutnya
54            head_8002 = head_8002.next_8002;
55
56            //prev pada head baru dibuat null
57
58        }
59    }
60 }

```

```

57     head_8002.prev_8002 = null;
58     }
59     System.out.println("lagu pertama berhasil dihapus");
60 }
61 //method mengembalikan playlist dari awal ke akhir
62 public void tampilLagu_8002() {
63
64     //cek playlist kosong
65     if(head_8002 == null) {
66         System.out.println("playlist kosong!");
67         return;
68     }
69     System.out.println("\n==Playlist Maju ==");
70
71     //variabel bantu untuk traversal
72     Lagu_2511538002 bantu_8002 = head_8002;
73
74     //mengambilkan data lagu
75     while (bantu_8002 != null){
76         System.out.println("judul"      : "+"
77             bantu_8002.getJudul_8002());
78         System.out.println("Penanyi"   : "+"
79             bantu_8002.getPenyanyi_8002());
80         System.out.println("-----");
81
82         bantu_8002 = bantu_8002.next_8002;
83     }
84 }
85
86 //method mengembalikan playlist dari akhir ke awal
87 public void tampilMundur_8002() {
88
89     //cek playlist kosong
90     if(tail_8002 == null) {
91         System.out.println("Playlist kosong!");
92         return;
93     }
94     System.out.println("\n==playlist Mudur==");
95     //traversal dari tail
96     Lagu_2511538002 bantu_8002 = tail_8002;
97     while(bantu_8002 != null) {
98         System.out.println("judul"      : "+"
99             bantu_8002.getJudul_8002());
100         System.out.println("Penanyi"   : "+"
101             bantu_8002.getPenyanyi_8002());
102         System.out.println("-----");
103
104         bantu_8002 = bantu_8002.prev_8002;
105     }
106 }
107
108 //method mencari lagu berdasarkan judul
109 public void cariLagu_8002(String judulCari_8002) {
110

```

```

111 //cek playlist kosong
112 if(head_8002 == null) {
113     System.out.println("Playlist kosong!");
114     return;
115 }
116 Lagu_2511538002 bantu_8002 = head_8002;
117 boolean ketemu_8002 = false;
118
119 //proses pencarian lagu
120 while(bantu_8002 != null) {
121
122     //pencarian tidak case-sensitiv
123     if(bantu_8002.getJudul_8002()
124         .equalsIgnoreCase( judulCari_8002)) {
125
126         System.out.println("\nLagu ditemukan!");
127         System.out.println("Judul : "+
128             bantu_8002.getJudul_8002());
129         System.out.println("Penyanyi : "+
130             bantu_8002.getPenyanyi_8002());
131
132         ketemu_8002 = true;
133         break;
134     }
135     bantu_8002 = bantu_8002.next_8002;
136 }
137 //jika lagu tidak ditemukan
138 if(!ketemu_8002) {
139     System.out.println("Lagu tidak ditemukan!");
140 }
141
142 //program utama
143 public static void main(String[] args) {
144     Scanner input_8002 = new Scanner(System.in);
145
146     //membuat obyek playlist
147     Musik_2511538002 playlist_8002 =
148         new Musik_2511538002();
149
150     int pilihan_8002;
151     do {
152         //tampilkan menu
153         System.out.println("\n==playlist Musi NIM: 2511538002");
154         System.out.println("1. Tambah Lagu");
155         System.out.println("2. Hapus Lagu pertama");
156         System.out.println("3. Lihat playlist (Maju)");
157         System.out.println("4. Lihat Playlist (Mundur)");
158         System.out.println("5. cari Lagu");
159         System.out.println("6. Keluar");
160         System.out.print("Pilihan : ");
161         pilihan_8002=input_8002.nextInt();
162         input_8002.nextLine();
163
164         switch (pilihan_8002) {
165             case 1:
166                 // input data lagu
167                 System.out.print("Judul Lagu : ");
168                 String judul_8002 = input_8002.nextLine();
169
170                 System.out.print("penyanyi : ");
171                 String penyanyi_8002 = input_8002.nextLine();
172
173                 //tambah lagu
174                 playlist_8002.tambahLagu_8002(
175                     judul_8002, penyanyi_8002);
176                 break;
177             case 2:
178                 // hapus lagu pertama
179                 playlist_8002.hapusLaguAwal_8002();
180                 break;
181             case 3:
182                 //tampil playlist Maju
183                 playlist_8002.tampilMaju_8002();
184                 break;
185             case 4:
186                 // tampil playlist mundur
187                 playlist_8002.tampilMundur_8002();
188                 break;
189             case 5:
190                 //cari lagu
191                 System.out.println("Masukan judul lagu : ");
192                 String cari_8002 = input_8002.nextLine();
193
194                 playlist_8002.cariLagu_8002(cari_8002);
195                 break;
196             case 6:
197                 System.out.println("program selesai.");
198                 break;
199             default:
200                 System.out.println("pilihan tidak valid");
201         }
202     } while (pilihan_8002 !=6);
203     input_8002.close();
204 }
205
206
207
208
209
210
211
212
213
214
215
216
217

```

2.2.4 Pejelasan pada kode Program Musik_2511538002

- Package dan import

```
package pekan6_2511538002;  
import java.util.Scanner;
```

package digunakan supaya bisa mengelompokkan file program agar lebih terstruktur. Menggunakan **import java.util.Scanner** untuk bisa mengambil input dari keyboard.

- Deklarasi Class

```
public class Musik_2511538002
```

Class Musik_2511538002 digunakan untuk mengelola playlist lagu menggunakan struktur data Double Linked List.

- Pointer Head dan Tail

```
Lagu_2511538002 head_8002;  
Lagu_2511538002 tail_8002;
```

Pointer ini digunakan agar traversal dapat dilakukan dari depan maupun belakang. Seperti head_8002 menunjuk node pertama pada playlist dan tail_8002 menunjuk node terakhir pada playlist.

- Constructor

```
public Musik_2511538002()  
head_8002 = null;  
tail_8002 = null;
```

Constructor digunakan untuk dapat menginisialisasi playlist saat pertama kali dibuat. Dan pada head dan tail sama dengan null, artinya playlist masih kosong dan belum memiliki node.

- Method tambahLagu_8002()

```
public void tambahLagu_8002(String judul_8002,
String penyanyi_8002) {
    Lagu_2511538002 laguBaru_8002 = new
    Lagu_2511538002(judul_8002, penyanyi_8002);
    if (head_8002 == null) {
        head_8002 = laguBaru_8002;
        tail_8002 = laguBaru_8002;
    } else {
        tail_8002.next_8002 = laguBaru_8002;
        laguBaru_8002.prev_8002 = tail_8002;
        tail_8002 = laguBaru_8002;
    }
    System.out.println("Lagu berhasil ditambahkan!");
}
```

Metode tambahLagu_8002() berfungsi untuk menambahkan lagu baru di akhir Double Linked List. Pada cara ini, program akan menghasilkan node baru yang memuat judul dan penyanyi dari lagu tersebut. Jika playlist belum memiliki isi, maka node baru akan menjadi kepala dan ekor. Namun, jika playlist sudah terisi data, maka node baru akan dihubungkan ke node terakhir lewat pointer next_8002 dan prev_8002, lalu posisi tail akan dipindahkan ke node baru itu

- Method hapusLaguAwal_8002()

```
public void hapusLaguAwal_8002() {
    if(head_8002 == null) {
        System.out.println("Playlist kosong!");
        return;
    }
}
```



```

}
if(head_8002 == tail_8002) {
    head_8002 = null;
    tail_8002 = null;
} else {
    head_8002 = head_8002.next_8002;
    head_8002.prev_8002 = null;
}
System.out.println("lagu pertama berhasil dihapus");
}

```

Metode `hapusLaguAwal_8002()` berfungsi untuk menghapus lagu yang pertama dalam playlist. Program terlebih dahulu mengecek apakah playlist kosong. Jika tidak ada isi, maka akan muncul pesan bahwa playlist tidak ada. Apabila hanya ada satu lagu, maka head dan tail akan diset menjadi null. Apabila ada lebih dari satu lagu, kepala akan dipindahkan ke node selanjutnya dan pointer `prev_8002` pada kepala yang baru dihapus sehingga tidak terhubung lagi dengan node sebelumnya

- Method `tampilMaju_8002()`

```

public void tampilMaju_8002() {
    if(head_8002 == null) {
        System.out.println("playlist kosong!");
        return;
    }
    System.out.println("\n===Playlist Maju ===");
    Lagu_2511538002 bantu_8002 = head_8002;
    while (bantu_8002 != null){
        System.out.println("judul    : "+

```

```

bantu_8002.getJudul_8002());
System.out.println("Penanyi  : "+
bantu_8002.getPenyanyi_8002());
System.out.println("-----");
bantu_8002 = bantu_8002.next_8002;
}
}

```

Metode tampilMaju_8002() berfungsi untuk menampilkan semua data lagu dari awal sampai akhir playlist. Traversing dimulai dari head_8002 dengan menggunakan variabel sementara, lalu program berpindah ke node berikutnya melalui pointer next_8002 hingga semua data lagu ditampilkan

- Method tampilMundur_8002()

```

public void tampilMundur_8002() {
if(tail_8002 == null) {
System.out.println("PlayList kosong!");
return;
}
System.out.println("\n===playlist Mudur===");
Lagu_2511538002 bantu_8002 = tail_8002;
while(bantu_8002 != null) {
System.out.println("judul  :"+
bantu_8002.getJudul_8002());
System.out.println("Penyanyi  :"+
bantu_8002.getPenyanyi_8002());
System.out.println("-----");
bantu_8002 = bantu_8002.prev_8002;
}
}

```

```
}
```

Metode tampilMundur_8002() digunakan untuk menyajikan data lagu dari bagian akhir ke bagian awal playlist. Perjalanan dimulai dari tail_8002, lalu bergerak ke belakang menggunakan pointer prev_8002 sampai seluruh informasi lagu berhasil ditampilkan. Metode ini menunjukkan keunggulan Double Linked List karena data dapat diakses dari kedua arah

- Method cariLagu_8002()

```
public void cariLagu_8002(String judulCari_8002) {  
    if(head_8002 == null) {  
        System.out.println("Playlist kosong!");  
        return;  
    }  
    Lagu_2511538002 bantu_8002 = head_8002;  
    boolean ketemu_8002 = false;  
    while(bantu_8002 != null) {  
        if(bantu_8002.getJudul_8002()  
            .equalsIgnoreCase( judulCari_8002)) {  
            System.out.println("\nLagu ditemukan!");  
            System.out.println("Judul : "+  
                bantu_8002.getJudul_8002());  
            System.out.println("Penyanyi : "+  
                bantu_8002.getPenyanyi_8002());  
            ketemu_8002 = true;  
            break;  
        }  
        bantu_8002 = bantu_8002.next_8002;  
    }  
}
```

```

if(!ketemu_8002) {
    System.out.println("Lagu tidak ditemukan!");
}
}

```

Metode cariLagu_8002() digunakan untuk menemukan lagu berdasarkan judul lagu yang dimasukkan oleh pengguna. Program akan menjelajahi playlist mulai dari head_8002 sampai node terakhir. Pencarian dilakukan dengan menggunakan equalsIgnoreCase() sehingga kapital dan non-kapital tidak memengaruhi hasil pencarian. Apabila lagu ditemukan, maka informasi mengenai lagu tersebut akan ditampilkan, sementara jika tidak ditemukan, program akan menunjukkan pesan bahwa lagu tidak ada

- Method main()

```

public static void main(String[] args) {
    Scanner input_8002 = new Scanner(System.in);
    Musik_2511538002 playlist_8002 =
    new Musik_2511538002();
    int pilihan_8002;
    do {
        System.out.println("\n==playlist Musi NIM:
        2511538002");
        System.out.println("1. Tambah Lagu");
        System.out.println("2. Hapus Lagu pertama");
        System.out.println("3. Lihat playlist (Maju)");
        System.out.println("4. Lihat Playlist (Mundur)");
        System.out.println("5. cari Lagu");
        System.out.println("6. Keluar");
    }
}

```

```
System.out.print("Pilihan : ");  
pilihan_8002=input_8002.nextInt();  
input_8002.nextLine();
```

Method `main()` adalah metode utama yang dipakai untuk menjalankan program daftar putar musik. Pada metode ini dibuat objek `playlist` dan menu interaktif dengan menggunakan perulangan `do-while` serta `switch-case`. Pengguna dapat memilih opsi seperti menambahkan lagu, menghapus lagu awal, menampilkan `playlist` ke depan, menampilkan `playlist` ke belakang, mencari lagu, dan keluar dari aplikasi

- Switch Case

```
switch (pilihan_8002) {
```

Digunakan untuk menjalankan fitur sesuai pilihan menu pengguna.

- Case 1 → tambah lagu
- Case 2 → hapus lagu pertama
- Case 3 → tampil `playlist` maju
- Case 4 → tampil `playlist` mundur
- Case 5 → cari lagu
- Case 6 → keluar program

2.2.5 Output kode program

```
===playlist Musi NIM: 2511538002
1. Tambah Lagu
2. Hapus Lagu pertama
3. Lihat playlist (Maju)
4. Lihat Playlist (Mundur)
5. cari Lagu
6. Keluar
Pilihan : 1
Judul Lagu : penolong yang setia
penyanyi : Melita Saputra
Lagu berhasil ditambahkan!
```

BAB III

PENUTUP

3.1 kesimpulan

Berdasarkan praktikum yang telah dilaksanakan, dapat disimpulkan bahwa Double Linked List merupakan struktur data yang menghubungkan node ke arah depan dan belakang sehingga penelusuran data menjadi lebih mudah. Praktikum ini mendukung pemahaman tentang cara menambahkan, mencari, dan menghapus data dalam Double Linked List.

Pada praktikum ini dipakai berbagai kelas seperti NodeDLL, InsertDLL, pencarian dan penghapusan untuk mempelajari operasi dasar dari double-linked list. Selain itu, ada juga tugas untuk membuat playlist musik dengan menggunakan kelas lagu dan musik yang mengaplikasikan konsep Double Linked List dalam pengelolaan data lagu.

Dengan praktikum ini, saya semakin mengerti bagaimana Double Linked List berfungsi dan penggunaannya dalam bahasa pemrograman Java. Di samping itu, praktikum ini juga meningkatkan keterampilan logika dan pemrograman dalam merancang program yang lebih terorganisir.

Daftar Pustaka

Rosa A.S dan M. Shalahuddin. 2018. Rekayasa Perangkat Lunak. Bandung: Teknologi Informasi.

Yadi, Suryadi. 2020. Data Struktur dan Algoritma dengan Pemrograman Java. Jakarta: Prenadamedia Group